

Project 2 (in C++): Given a file contains words (strings), the task is to write two sorting programs: Bubble-sort and Selection-sort to sort the input words in ascending order. Your program allows the user to pick which sorting algorithm to use for sorting. The task can be summarized as follows:

- 1) Open the input file; read and counts the number of words in the input file;
- 2) Dynamically allocate an 1D **string** array, size of count;
- 3) Close and re-open the input file and load the data in input file into the 1D array;
- 4) Ask the user to pick which sorting algorithm to sort: 1 for bubble-sort; 2 for selection-sort
- 5) Apply the selected sorting algorithm to the data in the array;
- 6) Output the sorted data to a file.

What you have to do:

1. Implement your program according to the specs given below.
2. Compile and debug your program until your program passes compilation.
3. You will be given two data files: data1 and data2; data1 contains a few words; data2 contains more words.
4. 1st run: pick Bubble-sort (enter 1 from console) , run your program with data1, then, eyeball the program's output for correctness. When your program produces correct output from data1, then run your program using data2.
5. 2nd run: pick Selection-sort (enter 2 from console), run your program with data1, then, eyeball the program's output for correctness. When your program produces correct output from data1, then run your program using data2.
6. 3rd run: enter 4, the program should exit without doing any sorting.

*** Include in your hard copy *DOC.docx file as follows:

- Cover page
- Source code
- inFile for data1 // with caption *** below is the input data1 ***
- inFile for data2 // with caption *** below is the input data2 ***

1st run:

- outFile for data1 // with caption *** The outFile of the Bubble-sort on data1 ***
- logFile for data1 // with caption *** The logFile of the Bubble-sort on data1 *** // limited to 3 pages, if more.
- outFile for data2 // with caption *** The outFile of the Bubble-sort on data2 ***
- logFile for data2 // with caption *** The logFile of the Bubble-sort on data2 *** // limited to 3 pages, if more.

2nd run:

- outFile for data1 // with caption *** The outFile of the Selection-sort on data1 ***
- logFile for data1 // with caption *** The logFile of the Selection -sort on data1 *** // limited to 3 pages, if more.
- outFile for data2 // with caption *** The outFile of the Selection -sort on data2 ***
- logFile for data2 // with caption *** The logFile of the Selection -sort on data2 *** // limited to 3 pages, if more.

Language: C++

Project Name: Bubble-sort and Selection-sort on strings.

Project points: 10 pts

Due Date: Soft copy (*.zip) and hard copies (*.docx):

(10/10 pts): on time, 9/19/2026, Saturday before midnight (11:59pm)

(-10/10 pts): non-submission, 9/19/2026, Saturday after midnight

*** Name your soft copy and hard copy files using the naming convention given in Project Submission Requirements.

*** All on-line submission MUST include Soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as below; otherwise, your submission will be rejected.

Email subject: (313) first name last name <Project 2: bubble-sort and Selection-sort on strings (C++)>

*** Inside the email body includes:

- Your answer to the 5 questions given in your email body. (-1 if not include.)
- Screen recoding link. (-1 if not submit screen recording).

I. Inputs: a) inFile (argv [1]): a file contains English words (strings), not in particular format. See data1 and data2.
b) from console: 1 for bubble-sort; 2 for selection sort

II. Output: a) outFile (argv [2]): a table with index and words of sorted words of input data (in ascending order.)
b) logFile (argv [3]): To log the progress of your program.

III. Data structures:

- Sort class:

int count;

string* dataAry; // 1D string array, size of count; need to dynamically allocate at run-time.

method:

- constructor (...)

- (int) countData (...) // re-use your countData(..) method from project1B,

- loadData (...) // re-use your loadData(..) method from project1B,

- bubbleSort(...) // **Algorithm is given below.**

- selectionSort(...) // **Algorithm is given below.**

- printDataAry (dataAry, fileOut) // Print a table (Index and data) of dataAry. On your own.

Index data

=====

0 John

1 Seth

2 Adam

3 Jess

:

IV. main (...)

Step 0: //similar to your Project1B

check argc countb(== 3) is correct and check to see if each file can be opened.

inFile ← open input file from argv [1] // i.e., ifstream inFile (argv [1])

outFile ← open from argv [2] // i.e., ofstream outFile (argv [2])

logFile ← open from argv [3] // i.e., ofstream logFile (argv [3])

step 1: count ← countData (inFile, logFile)

logFile ← "**** In main () count = " write count.

step 2: dataAry ← dynamically allocate, size of count.

step 3: close inFile

step 4: inFile ← open input file from argv [1] // re-open inFile

step 5: loadData (inFile, dataAry, count, logFile)

step 6: outFile ← "**** printing data before sorting. ****"

printDataAry (dataAry, outFile)

step 7: cout << " You have a choice of Bubble-sort or Selection-sort in the program. \n"

cout << "Enter 1 for Bubble-sort, enter 2 for Selection-sort.\n"

int whichSort

cin >> whichSort

step 8: if whichSort == 1

outFile ← "**** You select Bubble-sort ****"

bubbleSort (dataAry, count, logFile)

outFile << "**** Result of the Bubble-sort on the data****"

printDataAry (dataAry, outFile)

else whichSort == 2

```

        outFile ← “*** You select Selection-sort ***”
        selectionSort (dataAry, count, logFile)
        outFile << “*** Result of the Selection-sort on the data***”
        printDataAry (dataAry, outFile)
    else cout << “ Sorry! You must to enter either 1 for Bubble-sort or 2 for Selection-sort!\n”
step 9: delete[] dataAry
step 10: lose all files
*****
V. bubbleSort (dataAry, count, logFile)
*****
Step 0: logFile ← “*** Entering bubbleSort ()”
    (string) tmp
    (int) last ← count - 1
    (bool) swapFlag ← true
Step 1: i ← 0
    swapFlag ← false
Step 2: if dataAry[i] > dataAry[i+1]
    tmp ← dataAry [i]
    dataAry [i] ← dataAry [i+1]
    dataAry [i+1] ← tmp
    swapFlag ← true
Step 3: i++
Step 4: repeat step 2 to step 3 while i < last
Step 5: logFile ← “** content of array after one iteration of bubble sort, last = \n” // write last.
    printDataAry (dataAry, logFile) // print table to logFile.
Step 6: last ← last -1
Step 7: repeat step 1 to step 6 while swapFlag == true
Step 8: logFile ← “*** Leaving bubbleSort ()”
*****
VII selectionSort (dataAry, count, logFile)
*****
step 0: logFile ← “*** Entering selectionSort () ***”
step 1: i ← 0
step 2: string minVal ← dataAry[i]
    minIndex ← i
    j ← i+1
step 3: if dataAry[j] < minVal
    minIndex ← j
    minVal ← dataAry[j]
step 4: j++
step 5: repeat step 3 to step 4 while j < count
step 6: if minIndex != i
    string tmp ← dataAry [i]
    dataAry [i] ← dataAry [minIndex]
    dataAry [minIndex] ← tmp
step 7: logFile ← “** content of array after i-th iteration of selection sort \n” // write i.
step 8: printDataAry (dataAry, logFile) // print table to logFile.
step 9: i++
step 10: repeat step 2 to step 9 while i < count-1
step 11: logFile ← “*** Leaving selectionSort () ***”

```